# Reproducibility

# Reproducibility I

This section explains how to regenerate every artifact in the study from a clean checkout. The exemplar's reproducibility guarantee is structural: each result is produced by a tested function and a thin script, never transcribed by hand.

## How to regenerate everything I

From the repository root:

```
# 1. Run the analysis (writes figures + summary CSV,
prints output paths)
uv run python projects/templates/template_eda_noteboo
k/scripts/eda_analysis.py

# 1b. Regenerate the deterministic dataset sibling
(optional; stage 02 runs it too)
uv run python projects/templates/template_eda_noteboo
k/scripts/generate_measurements_data.py

# 2. Run the test suite with the coverage gate
uv run pytest
projects/templates/template_eda_notebook/tests \
    --cov=projects/templates/template_eda_notebook/sr
    c --cov-fail-under=90
```

## How to regenerate everything II

```
# 3. Render the manuscript
uv run python scripts/pipeline/stage_03_render.py
--project templates/template_eda_notebook
```

## Generated artifact registry I

The analysis scripts write the following artifacts under `projects/templates/template_eda_notebook/output/`:

# Generated artifact registry I

| Artifact | Produced by |
| --- | --- |
| `figures/height_histogram.png` | `histogram_data()` + analysis script |
| `figures/correlation_heatmap.png` | `correlation_heatmap_data()` + analysis script |
| `figures/group_counts.png` | `group_count_data()` + analysis script |
| `data/summary_statistics.csv` | `summary_statistics()` + analysis script |
| `data/measurements_generated.csv` | `generate_measurements()` + generator script |

## Generated artifact registry I

The `output/` tree is disposable and regenerated on every run; it is not the source of truth.

# Determinism I

- The dataset (`data/measurements.csv`) is a static, committed fixture with fixed content — the same file is read on every run, so every statistic is reproducible bit-for-bit.
- The figure-data preparers are pure transforms with no RNG; the same inputs always produce the same bin counts, correlation values, and group counts.
- `clean_dataset()` reports exactly how many rows it removed, so the complete-case row count is a checkable invariant.
- `generate_measurements()` (`src/template_eda_notebook/eda/generate.py`) regenerates a deterministic *sibling* of the dataset from a fixed NumPy seed: the same schema, 120 rows, the same missingness pattern, and the same correlation sign structure. The original fixture's exact random draw order is not recoverable, so the generator deliberately reproduces the fixture's documented contract rather than a byte-exact clone.

## Verification (no hand-transcribed numbers) I

Every quantitative claim in sec. 4 is either reproduced by running the analysis script or registered in `data/claim_ledger.yaml` for evidence-registry validation. The manuscript intentionally does not embed volatile values, so prose and artifacts cannot disagree. The notebook itself is verified structurally by `tests/test_notebook.py`, which confirms it is valid nbformat, that its `from src` imports resolve to the library's public surface, and that no cell defines its own logic.